



WEB DEVELOPMENT

Backend Practice

Flask → FastAPI → Deploy

Hyunwoo Park

Graduate School of Data Science

Seoul National University

hyunwoopark@snu.ac.kr / <https://hyunwoopark.com>

Today's Workshop

What We'll Build:

- A Flask monolith — frontend + backend from one server
- Refactor to FastAPI with separate frontend
- Deploy to Vercel (live URL on the internet!)
- Extend features with AI (vibe coding)

Prerequisites:

- GitHub Codespaces (set up in previous weeks)
- GitHub Copilot active

Format: Live coding — follow along step by step

Reference repo: github.com/oksure/backend-workshop

Workshop Roadmap

After this session, you will have:

- Built a full-stack app **two different ways**
- Understood monolith vs. separated architecture
- A **live API** deployed on the internet
- Used AI to extend your app

4 Steps:

Flask Monolith — one server does everything (25 min)

Separate with FastAPI — modern architecture (25 min)

Deploy to Vercel — put it online (15 min)

Vibe Code with AI — extend & experiment (15 min)

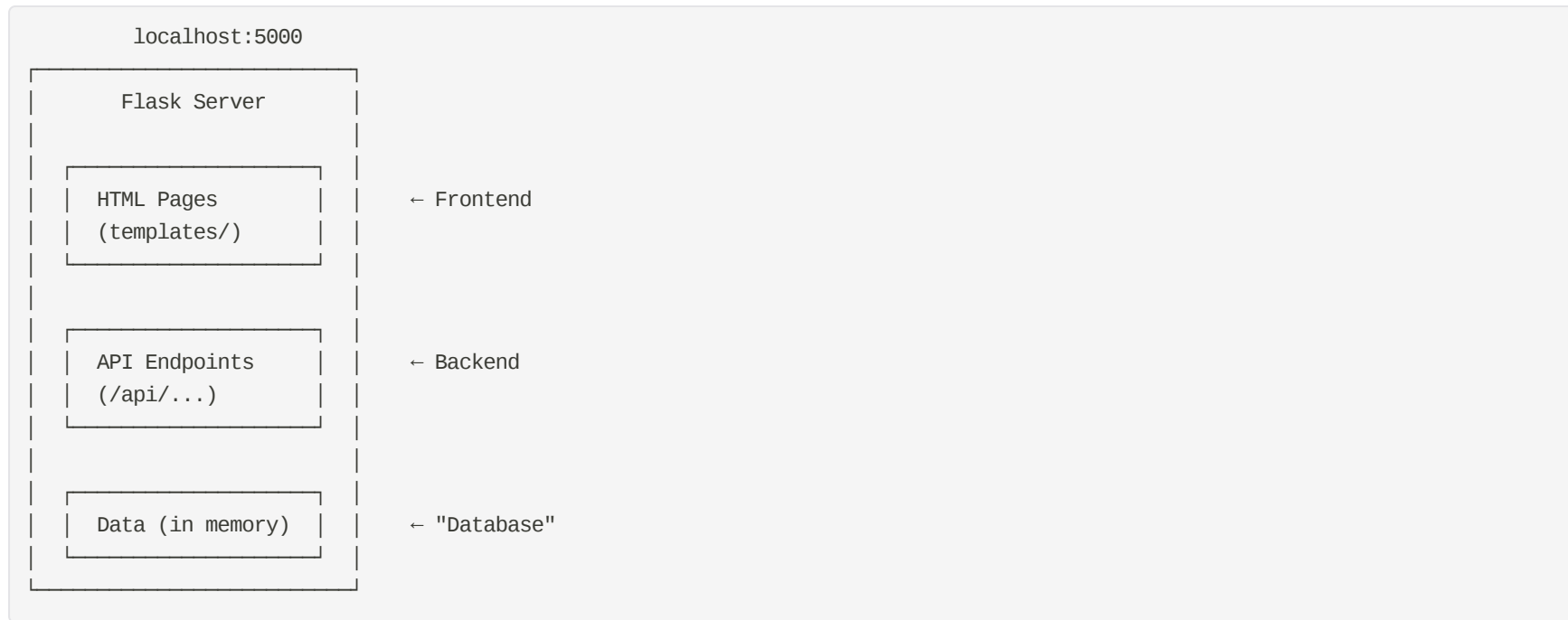


Step 1

Flask Monolith

One server serves everything

What's a Monolith?



One process. One port. Simple.

Monolith — Pros & Cons

Pros:

- Simplest possible setup
- No CORS issues (same origin)
- One thing to run, one thing to deploy
- Great for prototypes

Perfect for: Quick demos, small tools, learning the basics

Cons:

- Everything scales together
- One language for everything
- Deploy entire app for small changes
- Hard to split work across teams

Not great for: Large teams, production apps, mobile + web

Install Flask & Create App

In your Codespace terminal:

```
pip install flask
```

Create `app.py` :

```
from flask import Flask, render_template, request, jsonify

app = Flask(__name__)

# In-memory "database"
patients = []
next_id = 1

@app.route("/")
def home():
    return render_template("index.html")
```

Key Flask concepts:

- `Flask(__name__)` creates the app
- `@app.route("/")` maps URL to function

Add API Endpoints

Add to `app.py` :

```
@app.route("/api/patients")
def list_patients():
    return jsonify({"patients": patients, "count": len(patients)})

@app.route("/api/patients", methods=["POST"])
def create_patient():
    global next_id
    data = request.get_json()
    record = {
        "id": next_id,
        "name": data["name"],
        "age": data["age"],
        "department": data["department"],
        "notes": data.get("notes", "")
    }
    patients.append(record)
    next_id += 1
    return jsonify(record), 201
```

Flask is manual: You call `request.get_json()` and extract fields yourself. No automatic validation.

Create the HTML Template

Create the folder and file:

```
mkdir templates
```

Create `templates/index.html` — this is the frontend, served by Flask:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Patient Records</title>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body { font-family: system-ui, sans-serif; max-width: 800px;
      margin: 0 auto; padding: 2rem; background: #f5f5f5; }
    h1 { margin-bottom: 1.5rem; color: #1a1a1a; }
    .card { background: white; border-radius: 12px; padding: 1.5rem;
      margin-bottom: 1rem; box-shadow: 0 1px 3px rgba(0,0,0,0.1); }
    input, select { padding: 0.5rem; border: 1px solid #ddd;
      border-radius: 6px; font-size: 1rem; width: 100%; }
    button { padding: 0.5rem 1.5rem; background: #2563eb; color: white;
      border: none; border-radius: 6px; cursor: pointer;
      font-size: 1rem; margin-top: 0.5rem; }
```

HTML Template — Body

Continue in `templates/index.html`:

```
<body>
  <h1>Patient Records</h1>

  <div class="card">
    <h2>Add New Patient</h2>
    <form id="form" style="display:grid; gap:0.5rem; margin-top:1rem;">
      <input id="name" placeholder="Patient name" required>
      <input id="age" type="number" placeholder="Age" required>
      <select id="department" required>
        <option value="">Select department</option>
        <option value="cardiology">Cardiology</option>
        <option value="neurology">Neurology</option>
        <option value="oncology">Oncology</option>
        <option value="pediatrics">Pediatrics</option>
        <option value="emergency">Emergency</option>
      </select>
      <input id="notes" placeholder="Notes (optional)">
      <button type="submit">Add Patient</button>
    </form>
  </div>

  <div class="card">
    <h2>All Patients (<span id="count">0</span>)</h2>
```

JavaScript — Same Origin, No CORS!

Create `static/app.js`:

```
mkdir static
```

```
// No API URL needed — same server!
async function loadPatients() {
  const res = await fetch('/api/patients');
  const data = await res.json();

  document.getElementById('count').textContent = data.count;
  const el = document.getElementById('patients');
  el.innerHTML = data.patients.length === 0
    ? '<p style="color:#999; padding:1rem">No patients yet. Add one above!</p>'
    : data.patients.map(p => `
      <div class="patient">
        <strong>${p.name}</strong> (Age: ${p.age})
        <br>Department: ${p.department}
        ${p.notes ? `<br>Notes: ${p.notes}` : ''}
      </div>
    `).join('');
}

document.getElementById('form').onsubmit = async (e) => {
  e.preventDefault();
  await fetch('/api/patients', {
```

```
fetch('/api/patients') just works.
```

Same server = same origin = no CORS.

Run It!

```
flask run --host 0.0.0.0 --port 5000 --debug
```

In Codespaces:

- Click the popup "Open in Browser", or
- Go to **Ports** tab → port 5000 → click the globe icon

You should see:

- The patient form
- Add a patient → it appears in the list
- Everything from **one server** on **one port**

Try:

Add 2-3 patients

Refresh the page — patients are still there (same process!)

Full Flask App — 3 Files

```
flask-app/  
├─ app.py           # 25 lines – routes + logic  
├─ templates/  
│   └─ index.html   # HTML page served by Flask  
├─ static/  
│   └─ app.js        # Frontend JavaScript  
└─ requirements.txt  # flask
```

That's a full-stack web app. Flask serves the HTML, the same server handles API calls, JavaScript updates the page.

This is how web apps worked for 20 years — Django, Rails, PHP all follow this pattern.

But as apps grow, this model breaks down...



Step 2

Separate Frontend & Backend

The modern architecture

Why Separate?

The monolith gets heavy as your app grows:

- Frontend developer needs to touch Python files
- Backend developer needs to touch HTML templates
- Fix a typo in HTML → restart entire server → redeploy
- Mobile app needs the same data → duplicate API logic?
- Want React/Vue? → Jinja2 templates don't mix

The real world:

- Same API serves **web + mobile + CLI** apps
- Frontend on CDN (fast, global, cheap)
- Backend scales independently
- Teams work independently, communicate through API contract

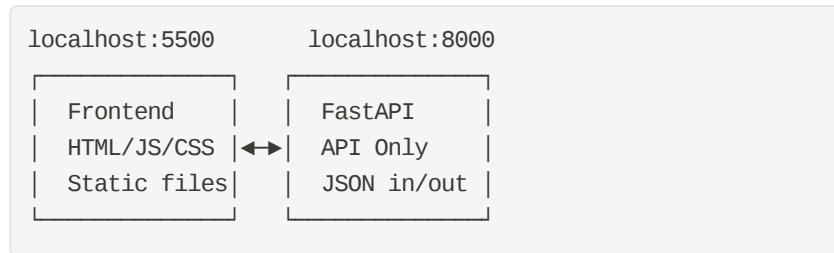
Monolith vs. Separated

Flask Monolith:



One process. One deploy. Simple but coupled.

FastAPI + Static Frontend:



Independent. Scalable. Team-friendly.

Frontend is just files. Any web server can host them.

Backend is just API. Any client can call it.

Switch to FastAPI

Install FastAPI:

```
pip install 'fastapi[standard]'
```

Create `server.py` :

```
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel

app = FastAPI()

# CORS – required when frontend and backend are on different ports
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

class Patient(BaseModel):
    name: str
    age: int
    department: str
```

FastAPI Endpoints

Add to `server.py` :

```
@app.get("/api/patients")
def list_patients():
    return {"patients": patients_db, "count": len(patients_db)}

@app.post("/api/patients", status_code=201)
def create_patient(patient: Patient):
    global next_id
    record = {"id": next_id, **patient.model_dump()}
    patients_db.append(record)
    next_id += 1
    return record

@app.get("/api/patients/{patient_id}")
def get_patient(patient_id: int):
    for p in patients_db:
        if p["id"] == patient_id:
            return p
    raise HTTPException(status_code=404, detail="Patient not found")
```

No HTML templates. No `render_template` . Just JSON in, JSON out.

Flask vs. FastAPI

Flask:

```
@app.route("/api/patients",
           methods=["POST"])
def create_patient():
    data = request.get_json()
    # manually extract fields
    record = {
        "name": data["name"],
        "age": data["age"],
        ...
    }
    return jsonify(record), 201
```

Manual everything.

FastAPI:

```
@app.post("/api/patients",
          status_code=201)
def create_patient(patient: Patient):
    # Pydantic validates automatically
    record = {
        "id": next_id,
        **patient.model_dump()
    }
    return record
```

Automatic validation + docs.

Visit `/docs` → interactive Swagger UI for free!

Separate Frontend

Create `index.html` (not in `templates/` — it's independent now):

Same HTML as before, but `app.js` changes:

```
// Point to the backend server
const API = "http://localhost:8000";

async function loadPatients() {
  const res = await fetch(`${API}/api/patients`);
  const data = await res.json();
  // ... same DOM updates
}

document.getElementById('form').onsubmit = async (e) => {
  e.preventDefault();
  await fetch(`${API}/api/patients`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ /* ... */ })
  });
  loadPatients();
};
```

Key change: `fetch('/api/...')` → `fetch(`${API}/api/...')`

Why CORS?

The browser blocks cross-origin requests by default:

```
Frontend (localhost:5500) → Backend (localhost:8000)  
Browser: "Different port = different origin. BLOCKED!"
```

That's why we added `CORSMiddleware` to FastAPI:

```
app.add_middleware(  
    CORSMiddleware,  
    allow_origins=["*"], # Allow all origins  
    allow_methods=["*"],  
    allow_headers=["*"],  
)
```

CORS = Cross-Origin Resource Sharing

This is a **browser** security rule, not a server rule. The server is happy to respond — the browser blocks it unless the server says "it's OK."

In Flask monolith: Same origin → no CORS needed. **Separated:** Different ports → CORS required.

Run Both Servers

Terminal 1— Backend (FastAPI):

```
uvicorn server:app --reload --host 0.0.0.0 --port 8000
```

Terminal 2 — Frontend (simple file server):

```
python -m http.server 5500
```

In Codespaces:

Go to **Ports** tab

Find port 8000 → copy the URL (e.g., `https://xxx-8000.app.github.dev`)

Update `API` in `app.js` with that URL

Open port 5500 in browser

Two terminals. Two servers. Connected via API.



Step 3

Deploy to Vercel

Put it on the internet

What is Vercel?

Vercel = Free deployment platform

- Hosts static files (frontend) on global CDN
- Runs Python functions (backend) as serverless
- Free tier, automatic HTTPS
- Auto-deploys on every `git push`

Serverless means: You write a function, Vercel runs it when someone calls your URL. No server to manage.

Project structure Vercel expects:

```
your-repo/
├─ api/
│   └─ index.py          # FastAPI → serverless function
├─ public/
│   └─ index.html        # Static frontend
│   └─ app.js
├─ requirements.txt
└─ vercel.json
```

Reorganize for Vercel

1. Create `api/index.py` — move your FastAPI code here:

```
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel

app = FastAPI(docs_url="/api/docs", openapi_url="/api/openapi.json")
app.add_middleware(CORSMiddleware, allow_origins=["*"],
                  allow_methods=["*"], allow_headers=["*"])

class Patient(BaseModel):
    name: str
    age: int
    department: str
    notes: str = ""

patients_db = []
next_id = 1

@app.get("/api/patients")
def list_patients():
    return {"patients": patients_db, "count": len(patients_db)}

@app.post("/api/patients", status_code=201)
def create_patient(patient: Patient):
```

Config Files

Create `vercel.json` :

```
{
  "rewrites": [
    { "source": "/api/(.*)", "destination": "/api/index" }
  ]
}
```

Create `requirements.txt` :

```
fastapi
```

Move frontend to `public/` :

```
mkdir -p public
cp index.html public/
cp app.js public/
```

Update `public/app.js` — change API URL:

```
const API = ""; // Empty string = same origin!
// const API = "https://api.example.com"; // Example API URL
```

Deploy!

1. Push to GitHub:

```
git init && git add . && git commit -m "Patient records app"  
gh repo create my-backend-app --public --source=. --push
```

2. Go to vercel.com:

- Sign in with GitHub
- "Add New Project"
- Import your repo
- Click **Deploy**

3. Wait 30-60 seconds...

4. Your app is live! `your-project.vercel.app`

Every `git push` **auto-deploys!**

Serverless Limitation

In-memory data resets between requests on Vercel.

Serverless = each request may run in a fresh process. `patients_db = []` starts empty.

This is fine for today's demo. The deployment flow is what matters.

For persistent data, add a database:

- MongoDB Atlas (free tier) — see `vercel-deploy-workshop` slides
- Supabase (free PostgreSQL)
- PocketBase

The architecture pattern stays the same — only the data layer changes.



Step 4

Vibe Code with AI

What is Vibe Coding?

"Vibe coding" = Describe what you want. AI writes the code.

Tools you already have:

- **GitHub Copilot** — inline suggestions in VS Code
- **Copilot Chat** — ask questions, generate code
- **Claude / ChatGPT** — paste code, ask for features

How it works:

Describe what you want in natural language

AI generates the code

You review, test, iterate

Ship it

The skill isn't writing syntax — it's describing what you want clearly.

Try These Prompts

In Copilot (type a comment, let it autocomplete):

```
# Add a DELETE endpoint that removes a patient by ID
```

```
# Add a search endpoint that filters patients by department and min_age
```

```
# Add a stats endpoint returning total, avg age, and count per department
```

In Copilot Chat or Claude:

"Add a delete button next to each patient in the frontend. When clicked, it should call `DELETE /api/patients/{id}` and refresh the list."

"Add a statistics section below the patient list showing total count, average age, and breakdown by department."

"Restyle the entire page using Tailwind CSS via CDN. Make it look modern."

Your Challenge (15 min)

Pick 1-2 features. Use AI to build them.

Delete patient — button + backend endpoint

Search/filter — by department or age range

Statistics — avg age, count by department

Better UI — Tailwind CSS, dark mode

Edit patient — update existing records

Sort — by name, age, or department

Tips:

- Paste your current code and ask AI to add the feature
- If it doesn't work, tell the AI what error you got
- Read the generated code — understand what it does

What You've Built Today

Step 1: Flask Monolith

One server serves HTML + API. Simple.

↓

Step 2: Separate with FastAPI

Frontend + Backend as independent pieces.

↓

Step 3: Deploy to Vercel

Live URL. Auto-deploy on push.

↓

Step 4: Vibe Code

AI-assisted feature development.

The progression:

- Flask → Easiest way to get something working
- FastAPI → Modern, validated, documented APIs
- Vercel → Ship to production in minutes
- AI → Accelerate development

Quick Reference — Flask

Run:

```
flask run --debug \  
--host 0.0.0.0 --port 5000
```

Serve HTML:

```
@app.route("/")  
def home():  
    return render_template("index.html")
```

API endpoint:

```
@app.route("/api/data",  
           methods=["POST"])  
def create():  
    data = request.get_json()  
    return jsonify(data), 201
```

Quick Reference — FastAPI

Run:

```
uvicorn server:app --reload \  
--host 0.0.0.0 --port 8000
```

API endpoint:

```
@app.post("/api/data",  
          status_code=201)  
def create(item: MyModel):  
    return item.model_dump()
```

Docs: `localhost:8000/docs`

CORS:

```
app.add_middleware(  
    CORSMiddleware,  
    allow_origins=["*"],  
    allow_methods=["*"],  
    allow_headers=["*"],  
)
```

Resources

Workshop code (clone and run):

- github.com/oksure/backend-workshop

Documentation:

- Flask — flask.palletsprojects.com
- FastAPI — fastapi.tiangolo.com
- Vercel Python — vercel.com/docs/functions/runtimes/python

Next session: Add a real database (MongoDB Atlas) for persistent storage



Thank you.

Backend Practice: Flask → FastAPI → Deploy

Hyunwoo Park

Graduate School of Data Science

Seoul National University

hyunwoopark@snu.ac.kr / <https://hyunwoopark.com>